Uncertainty-Driven Rule Induction for Dempster-Shafer Classifiers

Methodology and Experimental Design

Hayk Tarkhanyan

April 2026

Outline

1. Background: Dempster-Shafer Theory
2. DSGD++ Classifier
3. E1: Rule Source Comparison
4. E2: Iterative Uncertainty-Guided Refinement
5. E3: Multi-Source Rule Ensemble
6. E4: Pruning Pareto Frontier
7. E5: Baselines
8. MAF Initialization Comparison
9. Datasets and Setup
10. Summary

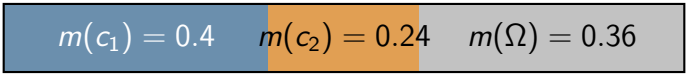
Mass Assignment Functions

Frame of discernment: $\Omega = \{c_1, c_2, \ldots, c_k\}$

A **Mass Assignment Function** distributes belief across subsets of Ω:

$$m : 2^{\Omega} \to [0, 1] \qquad \sum_{A \subseteq \Omega} m(A) = 1$$

For classification we use **singletons** + **full frame**:



$m(\Omega)$ represents uncertainty — belief not committed to any class.

Belief and Plausibility

From the MAF we derive two measures for any class c_j:

$$\mathrm{Bel}(c_j) = m(c_j) \qquad \mathrm{Pl}(c_j) = m(c_j) + m(\Omega)$$

Interpretation: The true probability of class c_j lies in:

$$P(c_j) \in [\mathrm{Bel}(c_j),\ \mathrm{Pl}(c_j)]$$

Example: With $m(+) = 0.4$, $m(-) = 0.24$, $m(\Omega) = 0.36$:

- $P(+) \in [0.40,\ 0.76]$ — likely positive but uncertain
- $P(-) \in [0.24,\ 0.60]$ — less likely but plausible

The **width** of the interval reflects model uncertainty.

Dempster's Rule of Combination

Combines **independent** evidence sources m_1 and m_2:

$$m_{1\oplus 2}(A) = \frac{1}{1-K} \sum_{B \cap C = A} m_1(B) \cdot m_2(C)$$

where $K = \sum_{B \cap C = \emptyset} m_1(B) \cdot m_2(C)$ measures conflict.

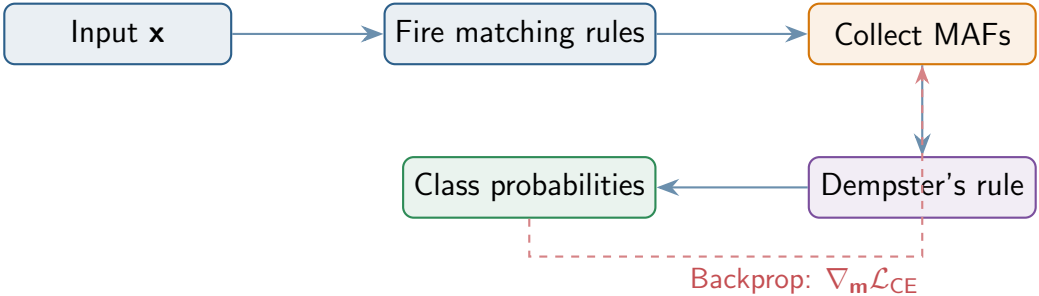
Key properties:

- **Commutative:** order of sources doesn't matter
- **Associative:** can chain multiple sources
- **Identity:** combining with $m(\Omega) = 1$ returns the other source unchanged

Agreeing sources $\rightarrow$ stronger belief. Conflicting sources $\rightarrow$ higher K.

DSGD++ Overview

Idea: Each rule is an evidence source with a learnable MAF.



- **Rules** are fixed Boolean predicates (not learned)
- **MAFs** are the learnable parameters (optimized by Adam)
- After each step: `normalize()` ensures $m_i \geq 0$ and $\sum m_i \leq 1$

Forward Pass: Commonality Trick

Direct computation of Dempster's rule is expensive.

Solution: Convert masses to *commonalities* and multiply.

For each fired rule i with masses $(m_{c_1}^{(i)}, \ldots, m_{c_k}^{(i)}, m_{\Theta}^{(i)})$:

$$q_{c_j}^{(i)} = m_{c_j}^{(i)} + m_{\Theta}^{(i)}$$

Combine all n fired rules by element-wise product:

$$Q_{c_j} = \prod_{i=1}^{n} q_{c_j}^{(i)}$$

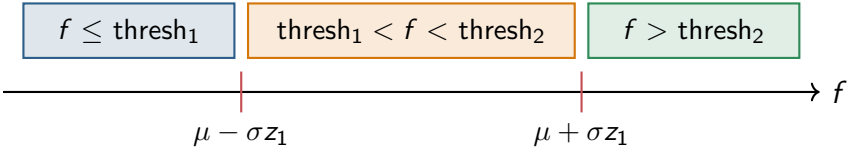
Normalize to get class probabilities:

$$P(c_j|\mathbf{x}) = \frac{Q_{c_j}}{\sum_{j'} Q_{c_{j'}}}$$

Base Rule Generation

For each feature f, compute mean μ_f and std σ_f on training data.

With $B = 2$ breaks at normal quantile positions z_1, z_2, generate 3 rules:



Total: $d \times (B + 1)$ rules for d features.

Example: 13 features $\times$ 3 rules $=$ 39 base rules for heart disease.

MAF Initialization

How do we set the initial masses before training?

Random (default):

- Small random mass per class, high uncertainty ($m_\Theta \approx 0.8$)
- Lets gradient descent learn everything from scratch

Clustering-based:

- Run KMeans on the training data
- For each rule, measure how close covered samples are to cluster centroids
- Initialize m_{c_j} proportional to cluster membership confidence
- Gives the optimizer a better starting point

Both are then refined by gradient descent — the initialization affects convergence speed but the final accuracy is often similar.

E1: Motivation

Base rules (single-feature thresholds) have a fundamental limitation:

They cannot capture feature interactions.

A rule like "glucose > 140" is useful, but the combination "glucose > 140 **AND** BMI > 30" is far more predictive for diabetes.

Research question:
If we mine conjunctive rules from external algorithms and add them to DSGD++, which mining source helps the most?

E1: Three Rule Miners

SkopeRules (bagged stumps):

- Trains many shallow decision trees, extracts rules meeting precision/recall thresholds
- Produces many diverse conjunctive rules

RIPPER (sequential covering):

- Greedy algorithm: find best rule, remove covered samples, repeat
- Produces compact, non-overlapping rule sets

Decision Tree (leaf-path extraction):

- Train a shallow tree (depth 3–4), extract root-to-leaf paths as rules
- Rules have good coverage and purity guarantees

E1: Protocol

1. Generate base single-feature rules ($d \times 3$ rules)
2. For each miner separately:
   a. Fit the miner on training data
   b. Extract rules as strings (e.g., `''X[2] > 0.5 and X[5] <= 1.3''`)
   c. Parse and convert to DSRule lambda functions
   d. Add to DSGD++ model with random MAF initialization
3. Train DSGD++ (500 epochs, cross-entropy, Adam)
4. Evaluate on held-out test set (accuracy, F1, AUC)

Baseline: DSGD++ with only base rules (no mined rules).

E2: Core Idea

Use DSGD++'s own uncertainty to guide rule mining.

After training, some rules have high m_Θ — the model is uncertain about the samples those rules cover.

Instead of mining rules globally, we mine rules **specifically for** the uncertain regions.

Then we retrain and check if uncertainty decreased.

This creates a **closed feedback loop**: uncertainty drives rule creation, which reduces uncertainty.

E2: Rule Confidence Score

After training, each rule r has learned masses $\mathbf{m}_r = (m_{c_1}, \ldots, m_{c_k}, m_\Theta)$.

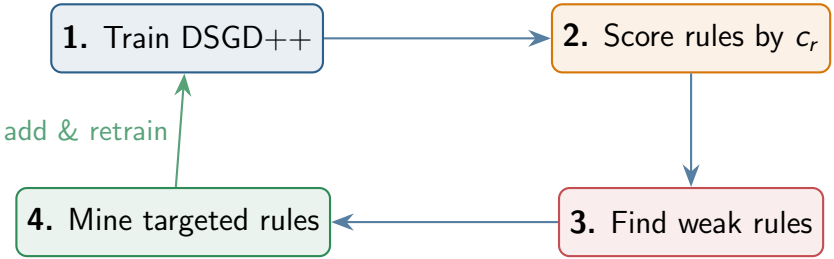
Confidence score:

$$c_r = \underbrace{\max_j \; m_{c_j}}_{\text{strongest class belief}} \times \underbrace{(1 - m_\Theta)}_{\text{certainty}}$$

- $c_r \approx 1$: rule is confident and informative
- $c_r < 0.3$: rule is **weak** — covers an ambiguous region

Example: $\mathbf{m} = (0.15, 0.10, 0.75) \implies c_r = 0.15 \times 0.25 = 0.04$ (very weak)

E2: The Iterative Loop



all strong? → **stop**

Maximum 3 iterations. Stop early if no weak rules remain.

E2: Targeted Mining Details

For each weak rule r (sorted by confidence, weakest first):

1. **Collect covered samples:** $S_r = \{\mathbf{x} : r(\mathbf{x}) = \text{True}\}$
2. **Skip if trivial:**
   - $|S_r| < 10$ (too few samples to mine from)
   - Region already pure (single class — nothing to improve)
3. **Run miner on S_r only** (not on full dataset)
4. **Add up to 3 new rules** per weak rule

Process up to 10 weak rules per iteration.

E2: Rule Cap and Safeguards

Problem: Iterative mining could explode the rule count.

Solution: Global rule cap at $3\times$ base rules.

- Example: 13 features $\times$ 3 = 39 base rules $\rightarrow$ cap at 117
- Once cap is reached, no more rules are added even if weak rules remain

Additional safeguards:

- Max 3 iterations (prevents infinite loops)
- Max 3 new rules per weak rule (prevents single-region flooding)
- Retraining after each iteration (all masses re-optimized jointly)
- Early stop if no new rules could be mined

E3: Motivation

E1 showed different miners excel on different datasets.

Can we combine rules from all miners at once?

In traditional ML, ensembles need careful weighting or stacking.

In DSGD++, we have a natural solution:

Dempster's rule was designed to combine evidence from heterogeneous sources.

Each miner's rules get their own MAFs — the model learns how much to trust each source during training.

E3: Protocol

1. Generate base single-feature rules
2. Add rules from **all** miners simultaneously:
   - SkopeRules (up to 10 rules)
   - RIPPER (up to 10 rules)
   - Decision Tree (depth 3, all leaf paths)
3. Each rule gets its own MAF — initialized randomly
4. Train DSGD++ jointly on the full rule set
5. Evaluate on held-out test set

Comparison: Single-source (base rules only) vs. multi-source.

No manual weighting needed — gradient descent handles it.

E4: Accuracy vs. Interpretability

More rules $\rightarrow$ potentially better accuracy, but less interpretable.

Goal: Find the sweet spot — how many rules can we remove while keeping accuracy?

Approach:

1. Train a full model with many rules (base + all mined)
2. Compute confidence c_r for every rule
3. Sweep a threshold τ from 0 to 0.45
4. For each τ: deactivate rules with $c_r < \tau$, measure accuracy
5. Plot: active rules vs. accuracy (Pareto frontier)

Baseline: Random pruning (remove random rules at same rates).

E4: Why Confidence Pruning Works

Rules with low c_r contribute little to decisions:

- High m_Θ means the rule's MAF is close to the identity element
- Combining with identity doesn't change the result
- Removing such rules has minimal effect on predictions

Confidence pruning vs. random pruning:

- Confidence pruning removes the least informative rules first
- Random pruning may accidentally remove important rules
- The gap between the two curves shows how much structure the confidence score captures

E5: Baseline Models

Interpretable baselines (fair comparison):

- Decision Tree (depth 4, depth 8)
- RuleFit — rules + linear model
- FIGS — fast interpretable greedy tree sums
- RIPPER — sequential covering
- SkopeRules — rule extraction from bagged trees
- Greedy Rule List — ordered if-then-else

Black-box baselines (upper bound on performance):

- Gradient Boosting (100 trees, depth 4)
- Random Forest (100 trees, depth 8)
- Logistic Regression

Same train/test split, same seed (509) across all models.

Random vs. Clustering Initialization

Question: Does a better starting point for MAFs lead to better results?

Random init:

- $m_{c_j} \sim$ small random values, $m_\Theta \approx 0.8$
- Model starts with high uncertainty, learns everything via gradient descent

Clustering init:

- KMeans on training data $\rightarrow$ cluster distances
- For each rule: measure how well covered samples align with clusters
- Initialize m_{c_j} proportional to cluster confidence
- Model starts closer to a good solution

We run E1–E3 with **both initializations** on all 6 datasets, saving results separately.

Datasets

Dataset	Samples	Features	Class ratio	Source
Heart Disease	303	13	54/46	UCI
Breast Cancer	683	9	65/35	UCI
Ionosphere	351	34	64/36	UCI
PIMA Diabetes	768	8	65/35	OpenML
QSAR Biodeg.	1,055	41	66/34	UCI
Phoneme	5,404	5	71/29	OpenML

- All binary classification, all numeric features
- 70/30 stratified train/test split, seed $= 509$
- StandardScaler fit on training data only

DSGD++ Configuration

Parameter	Value
Max epochs	500
Optimizer	Adam
Learning rate	0.005
Loss function	Cross-entropy
Statistical breaks	2 (3 rules per feature)
Rule cap (E2)	$3\times$ base rules
E2 max iterations	3
E2 uncertainty threshold	0.3
E2 max weak rules per iter	10
E2 max new rules per weak rule	3

Experiment Matrix

E1 — Which mined rules help the most?

↓

E2 — Does iterative uncertainty-driven mining outperform one-shot?

↓

E3 — Is multi-source combination better than any single miner?

↓

E4 — How many rules can we prune without losing accuracy?

↓

E5 — How does DSGD++ compare to existing methods?

Thank you

Questions?

Code: `github.com/HaykTarkhanyan/dst_research`